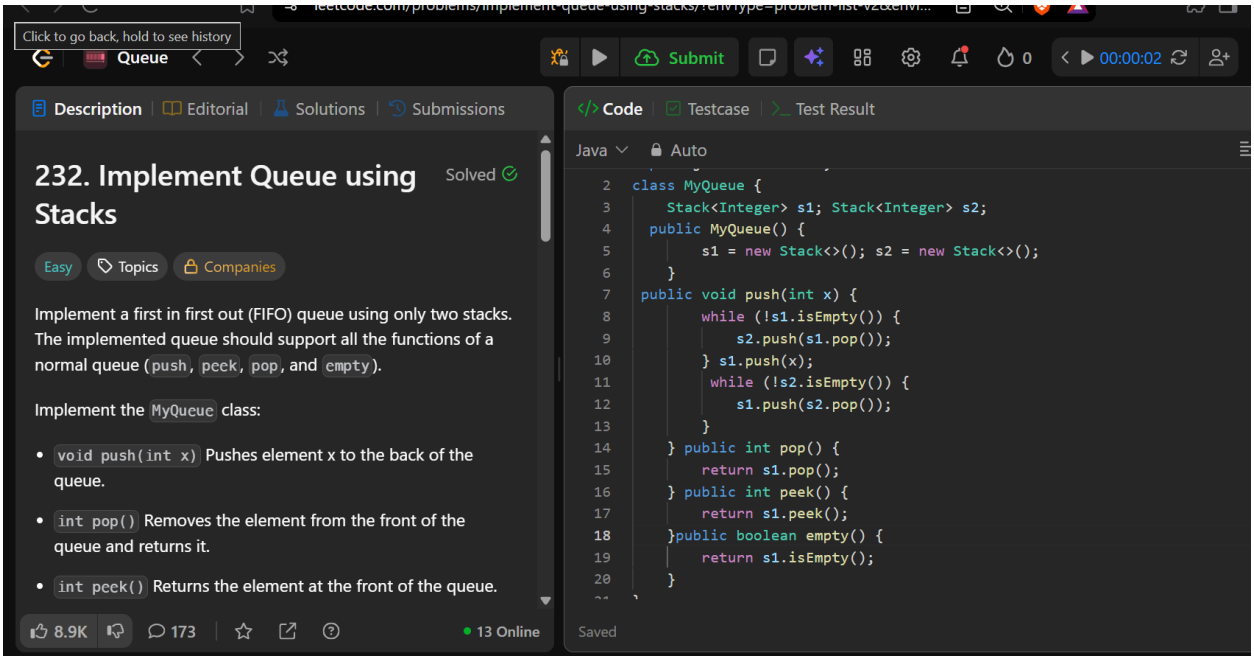
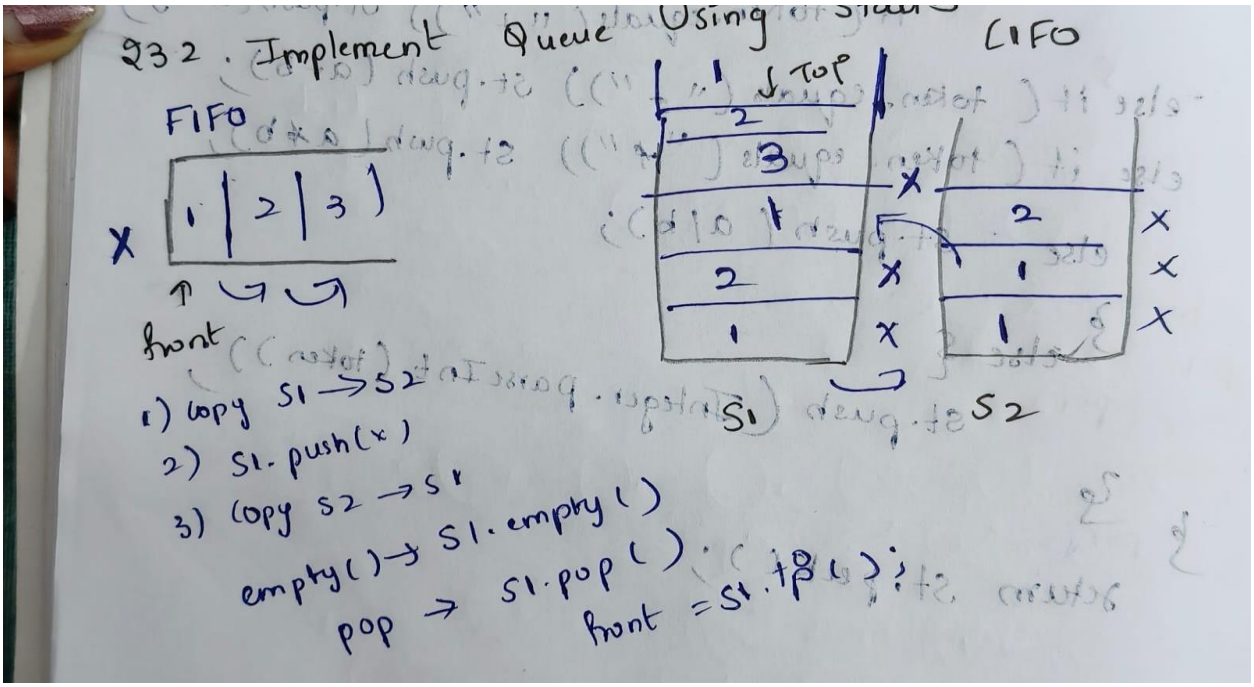
T. DHANA LAXMI

2505A41L46

1.implement queue using stack

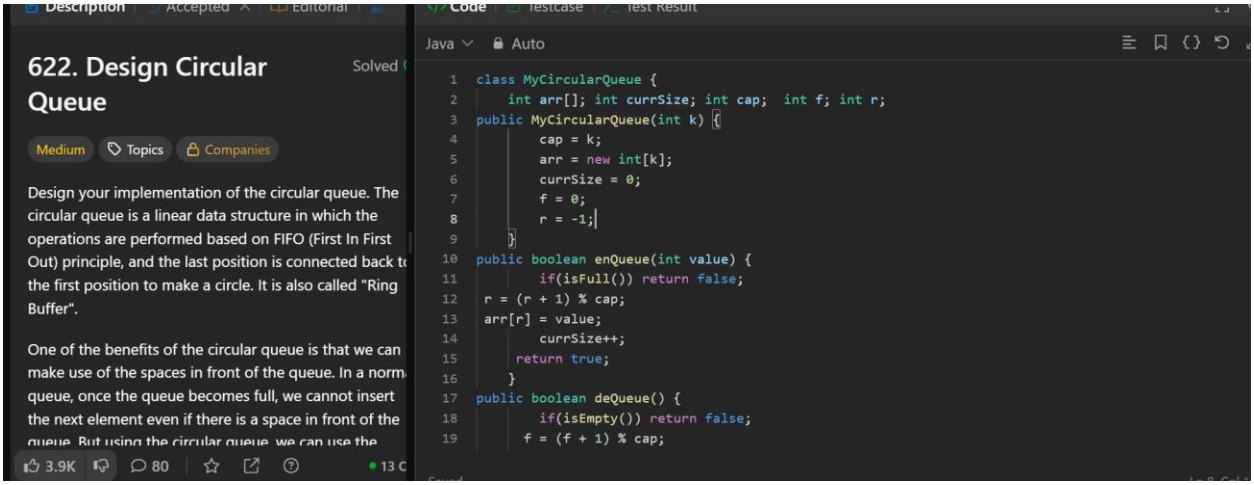
https://leetcode.com/problems/implement-queue-using-stacks/

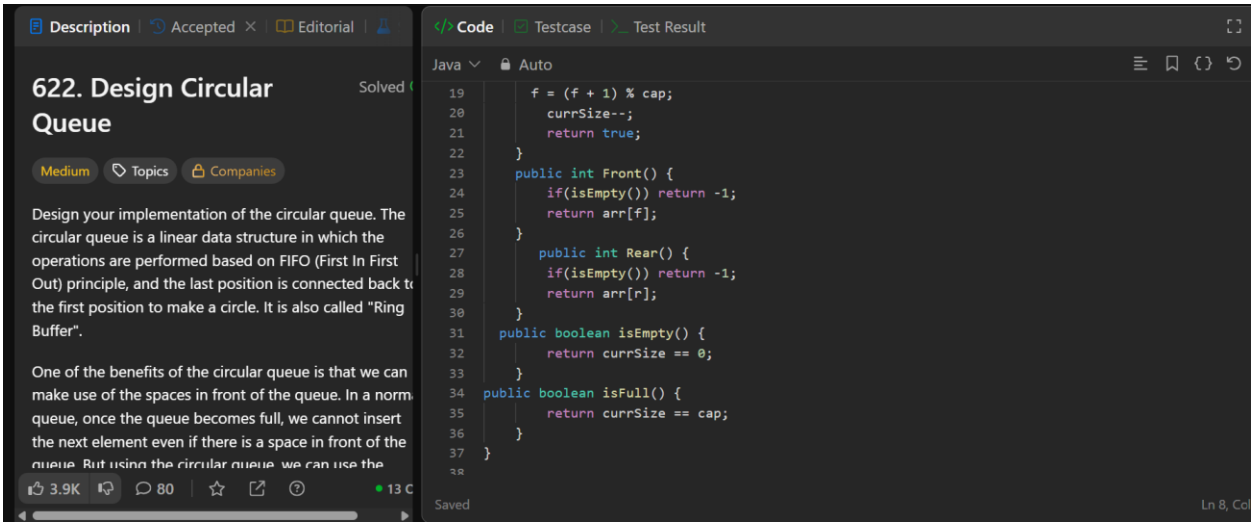


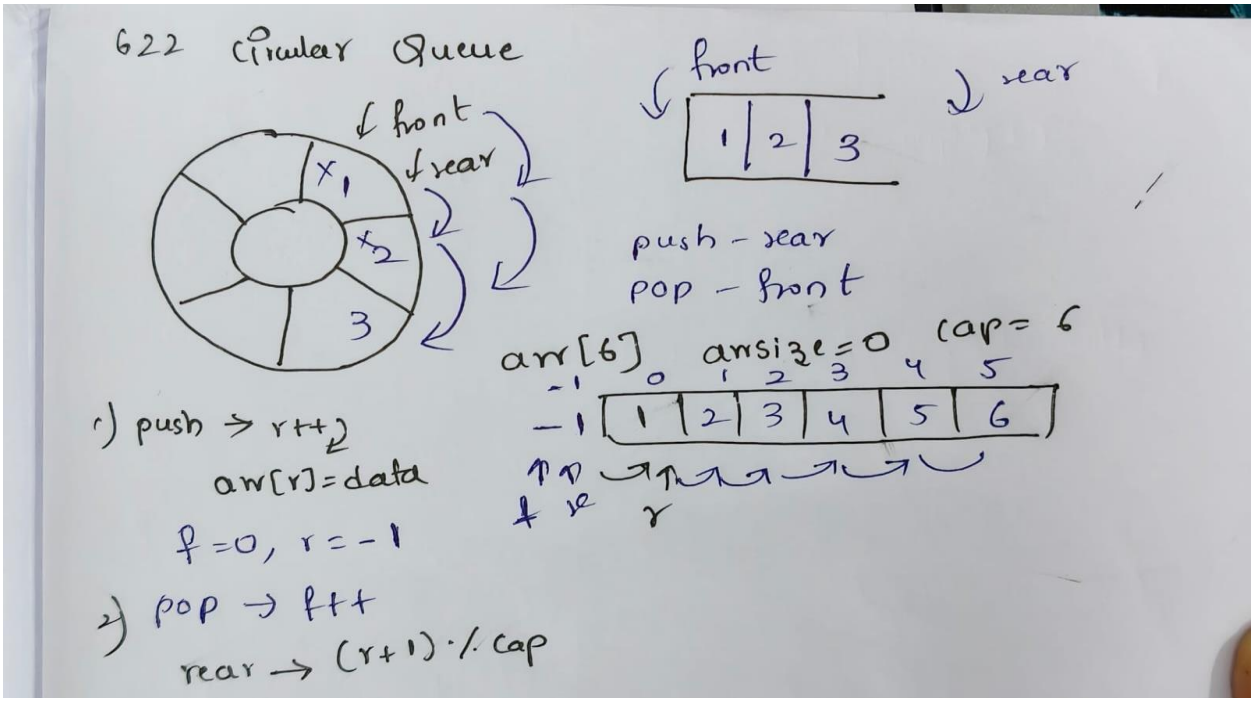


2.number of recent calls

https://leetcode. com/problems/number-of-recent-calls/


Description | Accepted | Editorial
622. Design Circular Queue
Solved
Medium
Topics
Companies
Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back t the first position to make a circle. It is also called "Ring Buffer".
One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a norm queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the
3.9K
80
Code | Testcase | Test Result
Java
Auto
class MyCircularQueue {
int arr[]; int currSize; int cap; int f; int r;
public MyCircularQueue(int k) {
cap = k;
arr = new int[k];
currSize = 0;
f = 0;
r = -1;
}
public boolean enQueue(int value) {
if(isFull()) return false;
r = (r + 1) % cap;
arr[r] = value;
currSize++;
return true;
}
public boolean deQueue() {
if(isEmpty()) return false;
f = (f + 1) % cap;


Description | Accepted | Editorial
622. Design Circular Queue
Solved
Medium
Topics
Companies
Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back t the first position to make a circle. It is also called "Ring Buffer".
One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a norm queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the
3.9K
80
Code | Testcase | Test Result
Java
Auto
f = (f + 1) % cap;
currSize--;
return true;
}
public int Front() {
if(isEmpty()) return -1;
return arr[f];
}
public int Rear() {
if(isEmpty()) return -1;
return arr[r];
}
public boolean isEmpty() {
return currSize == 0;
}
public boolean isFull() {
return currSize == cap;
}
}
Saved


622 Circular Queue
front
rear
x1
x2
3
front
rear
1
2
3
rear
push - rear
pop - front
arr[6]
arrsize=0
cap= 6
0 1 2 3 4 5
-1
1 2 3 4 5 6
f r
1) push → r++
arr[r]=data
f=0, r=-1
2) pop → f++
rear → (r+1) % cap

3.